

Daily Submission Report – Day 10

Date: 01 October 2026

Topic: AI Agents – Tools, Function Calling, Tool Selection, Validation, and Agent Workflows

What I learned:

Today I learned the fundamentals of AI Agents and how they are different from a normal chatbot. An AI Agent can understand a user's goal, make a decision, select an appropriate tool, execute the tool, observe the result, and provide a final response.

I learned about the following concepts:

- AI Agents
- Tools
- Function calling
- Tool selection
- Planning
- State
- Memory
- Agent workflows
- Multi-agent concepts
- Tool validation and safe execution

I also learned that an agent can use different tools depending on the user's request. For example, a room availability request can use an availability tool, while a price calculation request can use a price calculation tool.

What I implemented:

I created a **Hotel Assistant AI Agent** using Python and the Gemini API.

The agent was designed with two approved tools:

1. **check_room_availability()** – checks the availability of a specific room type.

2. **calculate_booking_price()** – calculates the estimated booking price based on the room type and number of nights.

The agent was configured to select the appropriate tool according to the user's request.

For example:

- "Is a Deluxe room available?" → `check_room_availability()`
- "How much is a Deluxe room for 3 nights?" → `calculate_booking_price()`

I also added input validation to prevent invalid requests, such as unsupported room types and invalid numbers of nights.

What worked:

The local tool functions were implemented successfully.

The room availability tool was designed to return the available number of rooms for supported room types.

The price calculation tool was designed to calculate the total price based on the room price and number of nights.

Validation was also implemented for:

- Invalid room types
- Zero or negative nights
- Excessive numbers of nights
- Unsupported actions

The agent structure also included restrictions so that only the approved hotel tools could be used.

What failed:

During Gemini API testing, the API initially returned a **503 UNAVAILABLE** error because the selected model was temporarily experiencing high demand.

After further testing, the API returned a **429 RESOURCE_EXHAUSTED** error.

The response indicated that the project's Gemini 3.8 Flash free-tier request quota had been exceeded. Because of this quota limitation, some agent requests could not be completed through the Gemini API.

How I solved it:

I investigated the API error messages and identified that the problem was related to the Gemini API quota rather than the Python tools themselves.

I separated the testing into two parts:

1. **Local tool testing** – testing the Python functions independently without calling Gemini.
2. **Agent/API testing** – testing Gemini's ability to select and use the tools.

I also changed the agent implementation to use a chat-based function-calling approach because the SDK provided a warning about directly using automatic function calling with `generate_content()`.

The API quota issue was identified as an external service limitation, so I avoided repeatedly sending requests and continued testing the tool logic locally.

What I would improve:

I would improve the agent by adding clearer tool-selection logging so that the selected tool, input parameters, tool result, and final response can be easily monitored.

I would also improve the project by:

- Adding more hotel-related tools.
- Adding stronger input validation.
- Adding structured logging.
- Adding automated tests for the tools.
- Adding short-term conversation state.
- Exploring memory for future interactions.
- Adding better error handling for API failures and quota limits.

- Exploring multi-agent architectures after gaining more experience with single-agent systems.

You: How much will a Deluxe room cost for 3 nights?

Agent:

A Deluxe room for 3 nights will cost ****\$360**** in total (\$120 per night).

`(.venv) PS E:\VisionRoot-AI engineer Internship\Github\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-10\practical-task> python agent.py`

=====

HOTEL ASSISTANT AI AGENT

=====

You: Is a Deluxe room available?

Direct use of automatic function calling (AFC) in `Models.generate_content` is not recommended. Instead, we recommend to use AFC in `Chat.send_message`. Similarly, direct use of AFC in `Models.generate_content_stream` is not recommended. Instead, we recommend to use AFC in `Chat.send_message_stream`.

Agent:

Yes, Deluxe rooms are available. Currently, there are 3 Deluxe rooms available.

Available capabilities:
- Check room availability
- Calculate booking price

Type 'exit' to quit.

You: How much is a Presidential room for 2 nights?

Agent Error:

429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-3.8-Flash\nPlease retry in 48.936228655s.'}, 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModelFreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-3.8-Flash'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '48s'}]}}

You: